

University of Asia Pacific
Department of Computer Science and Engineering
CSE 402: Operating System Lab

Lab Report: 07

Report Title: Disk Scheduling FCFS & SSTF

Submitted by:

Name **:** Md. Muhaimin Al Baki Raiyan

Student ID : 23101163

Section **:** C

Submitted to:

Atia Rahman Orthi

Lecturer,

Department of CSE, UAP

Date of Submission: 17/10/2026

1. Objective

The objectives of this lab are:

• To understand how the operating system decides the order of servicing disk I/O requests.
• To implement the FCFS and SSTF disk scheduling algorithms in Python.
• To calculate and compare the total head movement of both algorithms for the same request queue.

2. Theory

In a hard disk, the read/write head has to physically move to the cylinder where the data is stored. The time needed for this movement is called **seek time**, and it is the largest part of disk access time. When several requests are waiting, the order in which they are served decides how far the head has to travel. Disk scheduling algorithms try to choose a good order.

2.1 First Come First Serve (FCFS)

FCFS serves the requests exactly in the order they arrive in the queue. It is the simplest algorithm and every request is guaranteed to be served, but it does not care about the position of the head, so the head may jump back and forth across the disk.

2.2 Shortest Seek Time First (SSTF)

SSTF always serves the pending request that is nearest to the current head position. This greatly reduces head movement compared to FCFS, but requests far away from the head can be delayed for a long time if new nearby requests keep coming (*starvation*).

3. Problem Statement

Request Queue	3, 60, 10, 25, 30, 4
Initial Head Position	41

Using the data above, implement FCFS and SSTF, find the servicing sequence and the total head movement for each, and decide which algorithm is more efficient for this queue.

4. Algorithm

FCFS

1. Set *head* = initial position and *total* = 0.
2. For each request *r* in the queue (in given order): add |r − head| to *total*, then set head = r.
3. Print the sequence and *total*.

SSTF

1. Copy the queue into a list of pending requests; set *total* = 0.
2. While pending requests remain: find the request with minimum |r − head|, add that distance to *total*, move the head there and remove it from the list.
3. Print the sequence and *total*.

5. Source Code

```
requests=[3,60,10,25,30,4]
current_head=41
total_head_movement=0
seek_sequence=[]
pending_requests=list(requests)
print("SSTF=")
while pending_requests:
     closest_request=min(pending_requests,key=lambda req:abs(current_head-req))
     distance=abs(current_head-closest_request)
     total_head_movement+=distance
     current_head=closest_request
     seek_sequence.append(closest_request)
     pending_requests.remove(closest_request)
print(total_head_movement)
print(seek_sequence)
print("FCFS=")

total_head_movement=0
current_head=41
seek_sequence=[]
for request in requests:
    distance = abs(current_head - request)
    total_head_movement += distance
    seek_sequence.append(request)
    current_head=request
print(total_head_movement)
print(seek_sequence)
```

6. Output

```
Run   23101163_disk_scheduling
C:\Users\raiyan\PycharmProjects\OSLab> python 23101163_disk_scheduling.py

SSTF=
95
[30, 25, 10, 4, 3, 60]
FCFS=
191
[3, 60, 10, 25, 30, 4]

Process finished with exit code 0
```

Figure 1: Program output

7. Manual Calculation

The step-by-step head movement below verifies the program output.

FCFS

Step	From	To	Movement
1	41	3	38
2	3	60	57
3	60	10	50
4	10	25	15
5	25	30	5
6	30	4	26
		Total	**191**

SSTF

Step	From	To	Movement
1	41	30	11
2	30	25	5
3	25	10	15
4	10	4	6
5	4	3	1
6	3	60	57
		Total	**95**

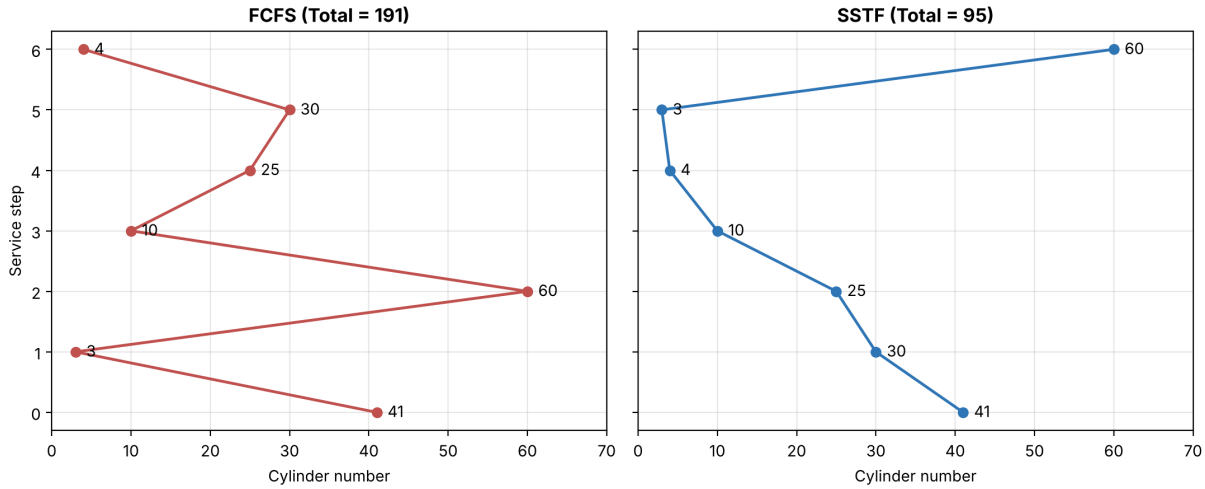


Figure 2: Head movement of FCFS (left) and SSTF (right)

8. Comparison: Which is better, SSTF or FCFS?

Criteria	FCFS	SSTF
Service sequence	$41 \to 3 \to 60 \to 10 \to 25 \to 30 \to 4$	$41 \to 30 \to 25 \to 10 \to 4 \to 3 \to 60$
Total head movement	191 cylinders	95 cylinders
Average seek per request	31.83	15.83
Starvation possible?	No	Yes

Answer: For this request queue, **SSTF is better** than FCFS.

Justification: FCFS follows the arrival order, and the queue keeps switching between low and high cylinders. Because of this the head makes two very long jumps, 3 → 60 (57) and 60 → 10 (50), which together are more than half of its total movement of 191 cylinders. SSTF avoids this by first clearing all the requests below the head (30, 25, 10, 4, 3) and then going to 60 only once at the end. As a result its total movement is just 95 cylinders, which is 96 cylinders (about 50.26%) less than FCFS. Less movement means less seek time, so

the queue is served faster.

However, SSTF is not fair to all requests. Cylinder 60 was the second request in the queue but was served last, and in a real system with continuously arriving nearby requests it could wait even longer. FCFS does not have this problem, but here it pays for fairness with almost double the head movement.

9. Discussion

In the SSTF part, the request list is first copied into *pending_requests*, so removing served requests does not change the original list that FCFS uses later. At every step, *min()* with a *key* of |current_head − request| directly gives the closest pending request. While testing, I noticed that if the request list is sorted before running, FCFS also follows the sorted order and gives 95 instead of 191, which is wrong because FCFS must keep the original arrival order. So the list is kept unsorted. Another observation is that SSTF only looks one step ahead: it went down to cylinder 3 and then had to travel 57 cylinders back up to 60. So SSTF is much better than FCFS here, but not always optimal. Algorithms such as SCAN and C-SCAN move the head in one direction at a time, which keeps movement low and also prevents starvation.

10. Conclusion

FCFS and SSTF disk scheduling were implemented successfully in Python for the queue [3, 60, 10, 25, 30, 4] with the head starting at 41. FCFS needed 191 cylinders of head movement and SSTF needed 95, so SSTF reduced movement by 96 cylinders and is the more efficient algorithm for this problem. FCFS remains useful for its simplicity and fairness, while SSTF is preferred when lower seek time is more important.